

Version Compatibility & Deprecation Windows, Worked Out by Hand

How many client×server version pairs actually interoperate, and exactly how long a deprecated feature is guaranteed to survive — counted on the real MCP timeline.

What this document does. An ecosystem with several client versions and several server versions has a *compatibility matrix*: each cell either interoperates or it doesn't. This document counts the matrix exactly using interval-overlap logic, then turns the 2026 deprecation policy (“at least one full revision deprecated before removal”) into calendar arithmetic with a guaranteed migration window.

Concepts covered: versions as a total order · support windows as intervals · interoperability = interval overlap · counting the $N \times M$ matrix · deprecation lifecycle · guaranteed migration window.

Internal learning material. v1.0

1 The setup: four revisions, totally ordered

MCP revisions are date strings, in strict order:

$$v_1 = 2024-11-05 < v_2 = 2025-03-26 < v_3 = 2025-06-18 < v_4 = 2026 \text{ (stateless)}.$$

We index them 1, 2, 3, 4. An implementation does not support a single version; it supports a contiguous **window** $[\ell, h]$ of revisions it can negotiate (it can speak anything from its oldest fallback ℓ to its newest h).

Intuition

Why a window and not a point? Negotiation (Module 1.1) picks $\max(V_c \cap V_s)$, so an implementation that can fall back stays compatible with older peers while preferring newer ones. “Which versions interoperate” therefore reduces to “do these two windows overlap” — a classic interval question.

2 Step 1 — Interoperability is interval overlap

Two endpoints with windows $[\ell_c, h_c]$ and $[\ell_s, h_s]$ can complete a handshake iff their windows overlap, i.e. they share at least one revision:

$$\text{interoperate} \iff \max(\ell_c, \ell_s) \leq \min(h_c, h_s).$$

When they do, the negotiated version is the top of the overlap, $v^* = \min(h_c, h_s)$.

Mini-example — this operation in isolation

Client window $[v_2, v_3]$, server window $[v_1, v_2]$: $\max(\ell) = \max(v_2, v_1) = v_2$ and $\min(h) = \min(v_3, v_2) = v_2$. Since $v_2 \leq v_2$, they overlap on exactly $\{v_2\}$ and negotiate $v^* = v_2$. Shift

the client to $[v_3, v_4]$: now $\max(\ell) = v_3 > \min(h) = v_2$ — no overlap, the handshake fails.

3 Step 2 — Counting the compatibility matrix

Take $N = 3$ client builds and $M = 3$ server builds with these support windows, and fill the $N \times M = 9$ cells with the negotiated v^* (or -- for no overlap):

client \ server	$S_a[v_1, v_2]$	$S_b[v_2, v_3]$	$S_c[v_3, v_4]$
$C_x[v_1, v_2]$	v_2	v_2	--
$C_y[v_2, v_3]$	v_2	v_3	v_3
$C_z[v_3, v_4]$	--	v_3	v_4

Count: 9 cells, 2 fail (the corners where windows are disjoint), so $7/9 \approx 77.8\%$ interoperate. The failures are exactly the pairs two windows apart: (C_x, S_c) and (C_z, S_a) .

Intuition

The interoperable region hugs the diagonal. Builds close in version overlap; builds two windows apart miss. This is why a “support the last two revisions” window keeps almost the whole matrix green: each endpoint stays reachable by everything within one step. Widen windows $\Rightarrow$ more green cells, at the cost of carrying old code.

Watch out

A green cell only guarantees a *handshake*, at version v^* . If v^* predates a feature you need (e.g. structured elicitation landed in v_3), the cell “works” but the feature is absent. Compatibility is about the negotiated floor, not about feature availability — check both.

4 Step 3 — Deprecation as a guaranteed window

Pre-2026, features vanished mid-revision (batching was removed in v_3 with one revision of overlap). The 2026 policy makes the window a guarantee: a feature slated for removal **must spend at least one full revision in the *deprecated* state**, with a published sunset. The lifecycle is four stages:

Introduced $\rightarrow$ Stable $\rightarrow$ Deprecated $\rightarrow$ Removed.

Let revision r_d be where a feature is first marked deprecated and r_x where it is removed. The policy is

$$r_x \geq r_d + 1 \quad (\text{at least one full revision of dual-support}).$$

Mini-example — the batching cautionary tale

Batching was **Stable** through v_2 (2025-03-26) and **Removed** in v_3 (2025-06-18) — effectively $r_x = r_d + 1$, the bare minimum. Servers that skipped one changelog broke that quarter. The post-2026 rule makes that minimum *mandatory and announced*, so the same removal would ship migration tooling and a sunset date one revision ahead.

Calendar arithmetic. Revisions land roughly every $\Delta \approx 4\text{--}8$ months. With the one-revision rule, the guaranteed migration window in wall-clock time is at least one cadence:

$$T_{\text{migrate}} \geq \Delta \approx 4\text{--}8 \text{ months.}$$

If your release train ships every 2 months, that is ≥ 2 of your own releases to migrate within — the policy exists precisely so the number is knowable in advance rather than “whenever it got removed.”

Watch out

“At least one revision” is a floor, not the actual lifetime. Don’t treat deprecation as “safe to ignore until it’s gone” — new code should migrate the moment a feature enters **Deprecated**, because the guarantee only protects you for one revision, and the next one may be the removal.

5 The argument, at a glance

#	Idea	Formula	Value
1	Interoperate iff windows overlap	$\max \ell \leq \min h$	—
1	Negotiated version	$v^* = \min(h_c, h_s)$	—
2	Matrix size	$N \times M$	9
2	Interoperable fraction	green / total	$7/9 = 77.8\%$
3	Deprecation rule	$r_x \geq r_d + 1$	≥ 1 revision
3	Guaranteed migration time	$\geq \Delta$	4-8 months

6 Equation cheat-sheet

Overlap test	$\max(\ell_c, \ell_s) \leq \min(h_c, h_s)$
Negotiated version	$v^* = \min(h_c, h_s)$
Matrix cells	$N \times M$
Deprecation floor	$r_x \geq r_d + 1$
Migration window (time)	$T_{\text{migrate}} \geq \Delta$ (revision cadence)

A Reference implementation

```
def negotiate(win_c, win_d):          # windows as (lo, hi) ints
    lo = max(win_c[0], win_d[0])
    hi = min(win_c[1], win_d[1])
    return hi if lo <= hi else None    # None = no overlap

clients = {"Cx":(1,2), "Cy":(2,3), "Cz":(3,4)}
servers = {"Sa":(1,2), "Sb":(2,3), "Sc":(3,4)}
green = sum(negotiate(c,s) is not None
             for c in clients.values() for s in servers.values())
print(green, "/", len(clients)*len(servers))    # 7 / 9

def migration_ok(r_dep, r_remove):
    return r_remove >= r_dep + 1          # 2026 policy floor
print(migration_ok(2, 3))                # True (batching: bare min)
```

— end of document —